

SIT744 Assignment 1 (T1 2026)

Backpropagation as Reverse Composition of Jacobians

Important Assessment Details

Weight: 20%

Format: Individual

Due: Week 3 Friday (see unit site)

Submission: Submit a single Jupyter notebook (.ipynb) named: SIT744_A1_<StudentID>.ipynb (Example: SIT744_A1_12345678.ipynb), containing all derivations, code implementations, written explanations, and passing automated checks.

1. Structural Overview

This assignment studies backpropagation as **reverse composition of Jacobians**.

We analyse a simple neural network as a composition of differentiable maps between finite-dimensional vector spaces.

1.1 Forward Composition

We study the composed map

$$L = \ell \circ g \circ \phi \circ f,$$

where the maps ℓ, g, ϕ, f are defined as follows:

$$f(X) = XW + b, \quad f : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times h},$$

where $X \in \mathbb{R}^{N \times d}$ is a batch of N inputs of d features and $W \in \mathbb{R}^{d \times h}$ and $b \in \mathbb{R}^h$ are weights and biases.

$$\begin{aligned}\phi(Z) &= \text{ReLU}(Z), & \phi : \mathbb{R}^{N \times h} &\rightarrow \mathbb{R}^{N \times h}, \\ g(H) &= Hv + c, & g : \mathbb{R}^{N \times h} &\rightarrow \mathbb{R}^N,\end{aligned}$$

where $v \in \mathbb{R}^h$ and $c \in \mathbb{R}$.

$$\ell(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad \ell : \mathbb{R}^N \rightarrow \mathbb{R}$$

Forward computation composes these maps left-to-right:

$$X \xrightarrow{f_{W,b}} Z \xrightarrow{\phi} H \xrightarrow{g_{v,c}} \hat{y} \xrightarrow{\ell} L(X)$$

1.2 Reverse-Mode Structure

Backpropagation computes gradients by applying the chain rule in reverse topological order.

For a differentiable map

$$f : \mathbb{R}^m \rightarrow \mathbb{R}^n,$$

its Jacobian at x is the matrix

$$J_f(x) \in \mathbb{R}^{n \times m}.$$

Forward vs Reverse

- **Forward-mode** propagates directional derivatives using Jacobian–vector products (JVP):

$$J_f(x) v$$

- **Reverse-mode** propagates gradients using vector–Jacobian products (VJP):

$$J_f(x)^\top \alpha$$

Reverse-mode therefore uses the **transpose of the Jacobian**.

Composition Rule

For composable maps,

$$(g \circ f)'(x) = J_g(f(x)) J_f(x).$$

Taking transposes gives

$$J_{g \circ f}(x)^\top = J_f(x)^\top J_g(f(x))^\top.$$

This is why:

- In a forward pass, we compose Jacobians left-to-right.
- In a backward pass, we multiply transposed Jacobians right-to-left.

Backpropagation as Reverse Composition

For our network,

$$L = \ell \circ g \circ \phi \circ f,$$

reverse-mode computes

$$\nabla L(X) = J_f(X)^\top J_\phi(Z)^\top J_g(H)^\top J_\ell(\hat{y})^\top 1.$$

Each stage implements one local vector–Jacobian product (VJP). The full gradient emerges by multiplying these local transposed Jacobians in reverse topological order.

Why Reverse-Mode is Efficient

Because the loss is scalar ($L : \mathbb{R}^p \rightarrow \mathbb{R}$), reverse-mode computes the entire gradient $\nabla L(X) \in \mathbb{R}^p$ in a single backward sweep. In contrast, forward-mode would require one sweep per parameter dimension. This efficiency is the structural reason backpropagation scales to large models.

2. Local Pullbacks (Building Blocks)

Backpropagation is implemented as a sequence of local vector–Jacobian products (VJPs). **This is the first section where implementation and derivation tasks begin.**

At each stage:

- You are given an upstream gradient.
- You compute gradients with respect to local inputs and parameters.
- You verify shapes carefully.
- You pass automated tests.

Each subsection below follows the same structure:

1. Concept
2. Derivation task
3. Implementation task
4. Verification

2.1 Pullback of the Loss (Seed Gradient)

Concept

The loss is scalar-valued:

$$\ell(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2.$$

Since $L(X) = \ell(\hat{y}, y)$, this is the starting point of reverse-mode.

We compute:

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial \ell(\hat{y}, y)}{\partial \hat{y}}.$$

This gradient becomes the **upstream seed** for all subsequent pullbacks.

Task

Task 2.1a — Derivation

Derive an explicit expression for

$$\frac{\partial L}{\partial \hat{y}}.$$

Your result should be written in vector form.

Task

Task 2.1b — Implementation

Implement:

`compute_dL_dyhat(y_hat, y)`

Requirements:

- Output shape must match $\hat{y}$.
- No loops.
- Must pass automated checks.

Automated Checks: Task 2.1b

```
import numpy as np

np.random.seed(0)
N = 7
y_hat = np.random.randn(N)
y = np.random.randn(N)

# student function
dL_dyhat = compute_dL_dyhat(y_hat, y)

# shape
assert isinstance(dL_dyhat, np.ndarray), "dL_dyhat must be a NumPy array"
assert (
    dL_dyhat.shape == y_hat.shape
), f"Shape mismatch: expected {y_hat.shape}, got {dL_dyhat.shape}"

# value (closed-form)
```

```

expected = (2.0 / N) * (y_hat - y)
assert (
    np.allclose(dL_dyhat, expected, rtol=1e-10, atol=1e-12)
), "Incorrect dL/dy_hat"
print("+++ Task 2.1b checks passed.")

```

2.2 Pullback of the Output Affine Map

Concept

In the local map

$$\hat{y} = Hv + c,$$

we have:

- $H \in \mathbb{R}^{N \times h}$
- $v \in \mathbb{R}^h$
- $c \in \mathbb{R}$
- $\hat{y} \in \mathbb{R}^N$

Given the upstream gradient

$$\frac{\partial L}{\partial \hat{y}} \in \mathbb{R}^N,$$

we compute gradients with respect to:

- H
- v
- c

Reverse-mode differentiation computes gradients by pulling back the upstream gradient through the local map:

$$\nabla_{\text{input}} L(X) = J^\top \nabla_{\text{output}} L(X).$$

Task

Task 2.2a — Derivation

Using perturbation reasoning, derive expressions for:

- $\frac{\partial L}{\partial H}$
- $\frac{\partial L}{\partial v}$
- $\frac{\partial L}{\partial c}$

Perturbation Strategy

To compute gradients of a local map in reverse-mode:

1. Compute the first-order variation of the local output.
2. Insert this variation into

$$\delta L = \left\langle \frac{\partial L}{\partial \hat{y}}, \delta \hat{y} \right\rangle.$$

3. Rearrange the expression so that each perturbation $(\delta H, \delta v, \delta c)$ appears independently.
4. The coefficient multiplying each independent perturbation is the corresponding gradient.

Gradients are therefore identified as **coefficients of independent variations**.

Step 1 — First-Order Variation of the Map

Because the map is affine,

$$\delta \hat{y} = \delta H v + H \delta v + \delta c.$$

This expression separates how perturbations in H , v , and c influence the output.

Step 2 — Propagate into the Loss

Write

$$\delta L = \left\langle \frac{\partial L}{\partial \hat{y}}, \delta \hat{y} \right\rangle.$$

Substitute the expression for $\delta \hat{y}$. You will obtain three terms corresponding to:

- a term involving δH ,
- a term involving δv ,
- a term involving δc .

Each must be handled separately.

Step 3 — Why Transposes Appear

To isolate each perturbation, use the identity

$$\langle a, Bx \rangle = \langle B^\top a, x \rangle.$$

This identity expresses how a linear map transfers from one argument of an inner product to the other. In reverse-mode differentiation, this transfer is exactly what produces the transpose of the local Jacobian.

Use this identity to rewrite terms so that:

- δH appears alone inside an inner product,
- δv appears alone,
- δc appears alone.

Only after this rearrangement can gradients be read off.

Step 4 — Reading Off the Gradients

After rearranging, your expression should take the structural form

$$\delta L = \langle (\text{expression}), \delta H \rangle + \langle (\text{expression}), \delta v \rangle + (\text{expression}) \delta c.$$

The expression multiplying each independent perturbation is:

- $\frac{\partial L}{\partial H}$
- $\frac{\partial L}{\partial v}$
- $\frac{\partial L}{\partial c}$

You must:

- Write these expressions explicitly,
- Verify their tensor shapes,
- Explain clearly where each transpose arises,
- Ensure dimensional consistency throughout.

Do not use memorised matrix-calculus formulas. All gradients must be derived from the perturbation expression and inner-product manipulation.

Task

Task 2.2b — Implementation

Implement:

`backward_output_affine(dL_dyhat, H, v)`

Must return:

- `dL_dH`
- `dL_dv`
- `dL_dc`

Requirements:

- Shapes must match exactly.
- No loops.
- Must pass automated checks.

Automated Checks: Task 2.2b

```
import numpy as np

def loss_from_yhat(y_hat, y):
    return np.mean((y_hat - y) ** 2)

def finite_diff_grad_H_v_c(H, v, c, y, eps=1e-6):
    N, h = H.shape

    # grads
    gH = np.zeros_like(H)
    gv = np.zeros_like(v)
    gc = 0.0

    base_yhat = H @ v + c
    base_L = loss_from_yhat(base_yhat, y)

    # H
    for i in range(N):
        for j in range(h):
            H_p = H.copy(); H_m = H.copy()
            H_p[i, j] += eps
```

```

        H_m[i, j] -= eps
        L_p = loss_from_yhat(H_p @ v + c, y)
        L_m = loss_from_yhat(H_m @ v + c, y)
        gH[i, j] = (L_p - L_m) / (2 * eps)

# v
for j in range(h):
    v_p = v.copy(); v_m = v.copy()
    v_p[j] += eps
    v_m[j] -= eps
    L_p = loss_from_yhat(H @ v_p + c, y)
    L_m = loss_from_yhat(H @ v_m + c, y)
    gv[j] = (L_p - L_m) / (2 * eps)

# c
L_p = loss_from_yhat(H @ v + (c + eps), y)
L_m = loss_from_yhat(H @ v + (c - eps), y)
gc = (L_p - L_m) / (2 * eps)

return gH, gv, gc

np.random.seed(1)
N, h = 5, 4
H = np.random.randn(N, h)
v = np.random.randn(h)
c = np.random.randn()
y = np.random.randn(N)

# upstream from Task 2.1 (but we compute here to avoid dependency)
y_hat = H @ v + c
dL_dyhat = (2.0 / N) * (y_hat - y)

# student function
dL_dH, dL_dv, dL_dc = backward_output_affine(dL_dyhat, H, v)

# shape checks
assert (
    dL_dH.shape == H.shape
), f"dL_dH shape mismatch: expected {H.shape}, got {dL_dH.shape}"
assert (
    dL_dv.shape == v.shape
), f"dL_dv shape mismatch: expected {v.shape}, got {dL_dv.shape}"

```

```

assert (
    np.isscalar(dL_dc)
    or (isinstance(dL_dc, np.ndarray) and dL_dc.shape == ())
), "dL_dc must be a scalar"

# value checks vs finite differences
gH_num, gv_num, gc_num = finite_diff_grad_H_v_c(H, v, c, y, eps=1e-6)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12))

errH = rel_err(dL_dH, gH_num)
errv = rel_err(dL_dv, gv_num)
errc = abs(dL_dc - gc_num) / (abs(dL_dc) + abs(gc_num) + 1e-12)

assert errH < 2e-5, f"dL_dH failed finite-diff check (rel err {errH:.2e})"
assert errv < 2e-5, f"dL_dv failed finite-diff check (rel err {errv:.2e})"
assert errc < 2e-5, f"dL_dc failed finite-diff check (rel err {errc:.2e})"
print("+++ Task 2.2b checks passed.")

```

2.3 Pullback of the Activation Map

Concept

Local map:

$$H = \phi(Z), \quad \phi = \text{ReLU}.$$

Elementwise definition:

$$H_{ij} = \begin{cases} Z_{ij} & Z_{ij} > 0 \\ 0 & Z_{ij} \leq 0 \end{cases}$$

Because the map is elementwise, its Jacobian is diagonal.

Given upstream gradient:

$$\frac{\partial L}{\partial H},$$

we compute:

$$\frac{\partial L}{\partial Z}.$$

Task

Task 2.3a — Derivation

Derive the pullback:

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial H} \odot \mathbf{1}_{Z>0}.$$

Here $\odot$ denotes elementwise (Hadamard) multiplication, and $\mathbf{1}_{Z>0}$ is the binary mask of the same shape as Z .

Task

Task 2.3b — Implementation

Implement:

`backward_activation_relu(dL_dH, Z)`

Requirements:

- Must be fully vectorised.
- No loops.
- Must pass automated checks.

Automated Checks: Task 2.3b

```
import numpy as np

def relu(Z):
    return np.maximum(Z, 0.0)

def loss_via_relu(Z, dL_dH):
    # Synthetic scalar objective: <dL_dH, H> where H = relu(Z)
    # This makes upstream gradient exactly dL_dH.
    H = relu(Z)
    return float(np.sum(dL_dH * H))
```

```

def finite_diff_grad_Z(Z, dL_dH, eps=1e-6):
    gZ = np.zeros_like(Z)
    it = np.nditer(Z, flags=["multi_index"], op_flags=["readwrite"])
    while not it.finished:
        idx = it.multi_index
        Z_p = Z.copy(); Z_m = Z.copy()
        Z_p[idx] += eps
        Z_m[idx] -= eps
        L_p = loss_via_relu(Z_p, dL_dH)
        L_m = loss_via_relu(Z_m, dL_dH)
        gZ[idx] = (L_p - L_m) / (2 * eps)
        it.iternext()
    return gZ

np.random.seed(2)
Z = np.random.randn(4, 6)
dL_dH = np.random.randn(4, 6)

# student function
dL_dZ = backward_activation_relu(dL_dH, Z)

# shape
assert (
    dL_dZ.shape == Z.shape
), f"Shape mismatch: expected {Z.shape}, got {dL_dZ.shape}"

# value vs finite differences (avoid kink exactly at 0 by nudging)
Z_safe = Z.copy()
Z_safe[np.abs(Z_safe) < 1e-3] += 1e-2

dL_dZ_num = finite_diff_grad_Z(Z_safe, dL_dH, eps=1e-6)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12))

err = rel_err(dL_dZ, dL_dZ_num)
assert err < 2e-5, f"Failed finite-diff check (rel err {err:.2e})"
print("+++ Task 2.3b checks passed.")

```

2.4 Pullback of the Input Affine Map

Concept

We now consider the affine map

$$Z = XW + b,$$

with shapes:

- $X \in \mathbb{R}^{N \times d}$
- $W \in \mathbb{R}^{d \times h}$
- $b \in \mathbb{R}^h$
- $Z \in \mathbb{R}^{N \times h}$

You are given the upstream gradient

$$\frac{\partial L}{\partial Z} \in \mathbb{R}^{N \times h}.$$

Your task is to compute gradients with respect to:

- W
- b
- X

using the perturbation framework developed in Task 2.2a.

Task

Task 2.4a — Derivation

Using inner-product-based perturbation reasoning,

$$\delta L = \left\langle \frac{\partial L}{\partial Z}, \delta Z \right\rangle,$$

derive explicit expressions for:

- $\frac{\partial L}{\partial W}$
- $\frac{\partial L}{\partial b}$
- $\frac{\partial L}{\partial X}$

Step 1 — First-Order Variation

Because the map is affine,

$$\delta Z = \delta X W + X \delta W + \delta b.$$

Here:

- $\delta X \in \mathbb{R}^{N \times d}$
- $\delta W \in \mathbb{R}^{d \times h}$
- $\delta b \in \mathbb{R}^h$

Note that δb is broadcast across the batch dimension: the same vector is added to every row of Z .

Step 2 — Isolate Independent Perturbations

Substitute δZ into

$$\delta L = \left\langle \frac{\partial L}{\partial Z}, \delta Z \right\rangle.$$

Then rearrange the expression so that:

- δX appears alone inside an inner product,
- δW appears alone,
- δb appears alone.

As in Task 2.2a, you will need to transfer linear maps across the inner product. Transpose operations arise from this adjoint relationship.

Step 3 — The New Structural Feature: Batch Accumulation

Unlike Task 2.2a, the bias $b \in \mathbb{R}^h$ is shared across all N samples. When substituting δb into δZ , each row of Z is perturbed identically. Therefore, when computing

$$\left\langle \frac{\partial L}{\partial Z}, \delta b \right\rangle,$$

contributions from all samples accumulate.

Your derivation must clearly show:

- why a summation over the batch dimension appears,
- along which axis the summation occurs,
- why the resulting gradient has shape $\mathbb{R}^h$.

Step 4 — Final Requirements

Your derivation must:

- Provide explicit formulas for the three gradients,
- Verify dimensional consistency,
- Clearly indicate where transposes arise,
- Show explicitly why the bias gradient involves a batch sum.

Do not use memorised matrix-calculus formulas. The result must follow directly from perturbation and inner-product manipulation.

Task

Task 2.4b — Implementation

Implement:

`backward_input_affine(dL_dZ, X, W)`

Must return:

- `dL_dW`
- `dL_db`
- `dL_dX`

Requirements:

- No loops.
- Exact shape consistency.
- Must pass automated checks.

Automated Checks: Task 2.4b

```
import numpy as np

def loss_from_Z(Z, dL_dZ):
    # Synthetic scalar objective: <dL_dZ, Z>
    # This makes upstream gradient exactly dL_dZ.
    return float(np.sum(dL_dZ * Z))

def finite_diff_grad_X_W_b(X, W, b, dL_dZ, eps=1e-6):
    N, d = X.shape
```



```

d_, h = W.shape
assert d_ == d

# base
def Z_of(X_, W_, b_):
    return X_ @ W_ + b_

gX = np.zeros_like(X)
gW = np.zeros_like(W)
gb = np.zeros_like(b)

# X
for i in range(N):
    for j in range(d):
        X_p = X.copy(); X_m = X.copy()
        X_p[i, j] += eps
        X_m[i, j] -= eps
        L_p = loss_from_Z(Z_of(X_p, W, b), dL_dZ)
        L_m = loss_from_Z(Z_of(X_m, W, b), dL_dZ)
        gX[i, j] = (L_p - L_m) / (2 * eps)

# W
for i in range(d):
    for j in range(h):
        W_p = W.copy(); W_m = W.copy()
        W_p[i, j] += eps
        W_m[i, j] -= eps
        L_p = loss_from_Z(Z_of(X, W_p, b), dL_dZ)
        L_m = loss_from_Z(Z_of(X, W_m, b), dL_dZ)
        gW[i, j] = (L_p - L_m) / (2 * eps)

# b
for j in range(h):
    b_p = b.copy(); b_m = b.copy()
    b_p[j] += eps
    b_m[j] -= eps
    L_p = loss_from_Z(Z_of(X, W, b_p), dL_dZ)
    L_m = loss_from_Z(Z_of(X, W, b_m), dL_dZ)
    gb[j] = (L_p - L_m) / (2 * eps)

return gX, gW, gb

```

```

np.random.seed(3)
N, d, h = 4, 5, 3
X = np.random.randn(N, d)
W = np.random.randn(d, h)
b = np.random.randn(h)

# arbitrary upstream gradient
dL_dZ = np.random.randn(N, h)

# student function
dL_dW, dL_db, dL_dX = backward_input_affine(dL_dZ, X, W)

# shape checks
assert (
    dL_dW.shape == W.shape
), f"dL_dW shape mismatch: expected {W.shape}, got {dL_dW.shape}"
assert (
    dL_db.shape == b.shape
), f"dL_db shape mismatch: expected {b.shape}, got {dL_db.shape}"
assert (
    dL_dX.shape == X.shape
), f"dL_dX shape mismatch: expected {X.shape}, got {dL_dX.shape}"

# value checks vs finite differences
gX_num, gW_num, gb_num = finite_diff_grad_X_W_b(X, W, b, dL_dZ, eps=1e-6)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12))

errX = rel_err(dL_dX, gX_num)
errW = rel_err(dL_dW, gW_num)
errb = rel_err(dL_db, gb_num)

assert errX < 2e-5, f"dL_dX failed finite-diff check (rel err {errX:.2e})"
assert errW < 2e-5, f"dL_dW failed finite-diff check (rel err {errW:.2e})"
assert errb < 2e-5, f"dL_db failed finite-diff check (rel err {errb:.2e})"
print("+++ Task 2.4b checks passed.")

```

3. Reverse Composition (Full Backpropagation)

In Section 2, you implemented each local pullback (VJP). Now we compose them in reverse topological order to compute the full gradient.

3.1 Conceptual Structure

Recall the forward composition:

$$L = \ell \circ g \circ \phi \circ f.$$

By the chain rule:

$$J_{g \circ f} = J_g J_f.$$

Taking transposes:

$$J_{g \circ f}^\top = J_f^\top J_g^\top.$$

Thus for the full network:

$$\nabla L = J_f^\top J_\phi^\top J_g^\top J_\ell^\top(1).$$

This is exactly backpropagation. At each stage:

1. Receives an upstream gradient.
2. Applies one local VJP.
3. Produces a new upstream gradient for the next stage.

3.2 Reverse Topological Order of Computation

Forward pass order:

1. $Z = f(X)$
2. $H = \phi(Z)$
3. $\hat{y} = g(H)$
4. $L = \ell(\hat{y})$

Backward pass order:

1. Compute $\frac{\partial L}{\partial y}$
2. Pull back through g
3. Pull back through ϕ
4. Pull back through f

The backward pass must proceed in the **exact reverse topological order** of the forward pass.

Reverse topological order refers to composing Jacobians in the opposite order of the forward computational graph.

Task

Task 3.1a — Explain the Reverse Topological Order

In at most 120 words, explain why reverse topological order is required when computing gradients on a computational graph. Your explanation should focus on node dependencies in a directed acyclic graph (DAG) and how local gradient contributions must be accumulated from downstream nodes. Do not appeal to matrix transposes or Jacobian multiplication—reason directly in terms of graph structure and data dependencies.

3.3 Implementation of Full Reverse Composition

You will now implement a complete forward + backward pass. Your function must:

1. Perform the forward pass.
2. Compute the loss.
3. Execute all pullbacks in reverse topological order.
4. Return the scalar loss and parameter gradients.

Task

Task 3.1b — Implementation

Implement:

`full_forward_backward(X, y, W, b, v, c)`

Your function must:

1. Perform forward computation.
2. Compute the scalar loss.
3. Execute pullbacks in reverse topological order:
 - Loss pullback
 - Output affine pullback

- Activation pullback
- Input affine pullback

4. Return:

- loss
- dW
- db
- dv
- dc

Requirements:

- No autograd libraries.
- No loops over samples.
- Must use your previously implemented functions.
- Must pass integration checks below.

Automated Checks: Task 3.1b

The full backward pass is verified against finite differences.

```
import numpy as np

def loss_only(X, y, W, b, v, c):
    Z = X @ W + b
    H = np.maximum(Z, 0.0)
    y_hat = H @ v + c
    return np.mean((y_hat - y) ** 2)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12))

np.random.seed(10)
N, d, h = 4, 5, 3

X = np.random.randn(N, d)
y = np.random.randn(N)

W = np.random.randn(d, h)
```

```

b = np.random.randn(h)
v = np.random.randn(h)
c = np.random.randn()

# student implementation
loss, dW, db, dv, dc = full_forward_backward(X, y, W, b, v, c)

# --- finite difference check ---
eps = 1e-6

def finite_diff_param(param, param_name):
    grad = np.zeros_like(param)
    it = np.nditer(param, flags=["multi_index"], op_flags=["readwrite"])
    while not it.finished:
        idx = it.multi_index
        params_p = dict(W=W.copy(), b=b.copy(), v=v.copy(), c=c)
        params_m = dict(W=W.copy(), b=b.copy(), v=v.copy(), c=c)
        params_p[param_name][idx] += eps
        params_m[param_name][idx] -= eps
        grad[idx] = (
            loss_only(X, y, **params_p) - loss_only(X, y, **params_m)
        ) / (2 * eps)
        it.iternext()
    return grad

# check W
dW_num = finite_diff_param(W, "W")
assert rel_err(dW, dW_num) < 2e-5, "dW failed finite-difference check"

# check b
db_num = finite_diff_param(b, "b")
assert rel_err(db, db_num) < 2e-5, "db failed finite-difference check"

# check v
dv_num = finite_diff_param(v, "v")
assert rel_err(dv, dv_num) < 2e-5, "dv failed finite-difference check"

# check c
c_p = c + eps
c_m = c - eps
dc_num = (

```

```

    loss_only(X, y, W, b, v, c_p) - loss_only(X, y, W, b, v, c_m)
    ) / (2 * eps)
assert (
    abs(dc - dc_num) / (abs(dc) + abs(dc_num) + 1e-12) < 2e-5
), "dc failed finite-difference check"

print("+++ Task 3.1b Full reverse composition checks passed.")

```

4. Verification via Gradient Checking

Reverse-mode differentiation computes gradients analytically via composition of Jacobians. To verify correctness, we compare these gradients to numerical approximations using finite differences. This does **not** derive gradients. It verifies that your implementation is correct.

4.1 Finite Difference Approximation

For a scalar-valued function

$$L(\theta),$$

the derivative with respect to one parameter θ_i can be approximated by:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon}.$$

This is the **central difference approximation**. It has truncation error of order ε^2 (See Bishop & Bishop 2024, Chapter 8).

4.2 Relative Error

To compare analytical and numerical gradients, we compute:

$$\text{rel error} = \frac{\|\mathbf{g}_{\text{analytic}} - \mathbf{g}_{\text{numeric}}\|}{\|\mathbf{g}_{\text{analytic}}\| + \|\mathbf{g}_{\text{numeric}}\| + 10^{-12}}.$$

A correct implementation should produce:

- relative error $\leq 10^{-5}$ for small problems.

Task

Task 4.1a — Conceptual Understanding

In at most 120 words, explain:

1. Why finite differences verify but do not derive gradients.
2. Why central difference is preferred over forward difference.
3. Why relative error is used instead of absolute error.

4.3 Implementation of Gradient Checking

You will now implement a reusable gradient checking function. This function should:

1. Select one parameter tensor (e.g., W).
2. Compute its analytical gradient via your full backward pass.
3. Compute its numerical gradient using finite differences.
4. Compute the relative error.

Task

Task 4.1b — Implementation

Implement:

`gradient_check(X, y, W, b, v, c, param_name)`

The function must:

- Compute analytical gradients using `full_forward_backward`.
- Compute numerical gradients using central differences.
- Return the relative error.

Requirements:

- Must work for at least parameter W .
- Must not use autograd libraries.
- Must be vectorised where possible.

Automated Checks: Task 4.1b

```
import numpy as np
```



```

np.random.seed(42)
N, d, h = 3, 4, 2

X = np.random.randn(N, d)
y = np.random.randn(N)

W = np.random.randn(d, h)
b = np.random.randn(h)
v = np.random.randn(h)
c = np.random.randn()

rel_error = gradient_check(X, y, W, b, v, c, param_name="W")

assert (
    rel_error < 2e-5
), f"Gradient check failed (rel error {rel_error:.2e})"
print("+++ Task 4.1b Gradient checking passed.")

```

5. Conceptual Reflection

In this assignment, you implemented backpropagation as reverse composition of Jacobians. This section asks you to reflect on the structure behind the computation.

5.1 Why Reverse-Mode Is Efficient

Task

Task 5.1 — Reverse-Mode Efficiency (150 words)

Explain:

1. Why reverse-mode computes the full gradient in one backward pass.
2. Why forward-mode would require multiple passes.
3. Why this makes reverse-mode practical for deep learning.

Your explanation must reference:

- Scalar-valued loss
- Jacobian–vector vs vector–Jacobian products
- Computational cost scaling

5.2 Why the Order Reverses

Task

Task 5.2 — Why Order Reverses (120 words)

Consider a composition of functions

$$f = f_L \circ \dots \circ f_1.$$

Using Jacobians, explain why reverse-mode differentiation applies linear maps in the reverse order. Your explanation must explicitly reference:

- Composition of Jacobians:

$$Df = Df_L \cdots Df_1$$

- The transpose identity:

$$(AB)^\top = B^\top A^\top$$

- Why reverse mode is computationally efficient when the output is scalar

Do **not** discuss graph traversal or topological ordering. Your explanation should be framed purely in terms of linear maps and Jacobian composition.

5.3 Conceptual Difficulty

Understanding reverse-mode requires internalising:

- Gradients as linear maps
- Transposes as adjoints
- Reverse composition of local VJPs
- The difference between computation graph and derivative graph

Task

Task 5.3 — Conceptual Challenge (120 words)

Describe what you found most conceptually challenging in this assignment. Be specific. For example,

- Why transposes appear
- Why bias gradients involve batch sums
- Understanding VJP vs JVP
- Tracking tensor shapes

Superficial answers will not receive full credit.

6 Final Verification Checklist

Before submitting your notebook, ensure the following:

6.1 Local Pullbacks (Section 2)

- ☐ All derivations are written clearly in Markdown.
- ☐ `compute_dL_dyhat` passes automated checks.
- ☐ `backward_output_affine` passes finite-difference checks.
- ☐ `backward_activation_relu` passes finite-difference checks.
- ☐ `backward_input_affine` passes finite-difference checks.

6.2 Full Reverse Composition (Section 3)

- ☐ `full_forward_backward` passes integration checks.
- ☐ Backward computation is performed strictly in reverse topological order.
- ☐ No autograd libraries are used.
- ☐ No loops over samples.

6.3 Gradient Checking (Section 4)

- ☐ `gradient_check` computes central-difference gradients.
- ☐ Relative error $\leq 1e-5$ for at least parameter W .
- ☐ Relative error formula is implemented correctly.

6.4 Written Components (Section 3–5)

- ☐ Reverse-order explanation (120 words).
- ☐ Finite-difference explanation (120 words).
- ☐ Reverse-mode efficiency explanation (150 words).
- ☐ Conceptual challenge reflection (120 words).

6.5 Final Integrity Check

- ☐ All automated checks run without assertion errors.
- ☐ Notebook executes top-to-bottom without manual intervention.
- ☐ File name follows required submission format.

Learning Verification

In cases where there is reasonable uncertainty regarding the authenticity or depth of demonstrated understanding, the Unit Chair may request a short verification interview (e.g., oral discussion or live walkthrough of the notebook).

The purpose of this discussion is to confirm that the submitted work reflects genuine attainment of the relevant learning outcomes. Failure to satisfactorily demonstrate understanding may result in further academic integrity review.

Marking Overview (100 Marks)

This assignment assesses both **technical correctness** and **structural understanding** of backpropagation as reverse composition of Jacobians.

1. Local Gradients (40 marks)

Strong submissions:

- Present clear derivations of each local pullback.
- Implement fully vectorised gradients with precise shape consistency.
- Demonstrate numerical correctness through automated and finite-difference checks.

2. Full Reverse Composition (20 marks)

High-quality work:

- Implements a correct forward pass.
- Executes backpropagation in strict reverse topological order.
- Uses the derived local pullbacks coherently.
- Explains why composition reverses in reverse-mode differentiation.

3. Gradient Verification (15 marks)

Effective implementations:

- Use central-difference approximation.
- Compute relative error appropriately.
- Show close numerical agreement ($\sim 1e-5$).

4. Conceptual Understanding (15 marks)

Insightful responses:

- Explain why reverse-mode is efficient for scalar losses.
- Connect Jacobian transposes to reversed composition order.
- Reflect thoughtfully on conceptual challenges encountered.

5. Technical Integrity (10 marks)

Submissions are expected to:

- Execute cleanly from top to bottom.
- Pass all provided checks.
- Follow the required format and tool constraints.

Performance Standard

High distinctions typically demonstrate:

- Numerically correct gradients,
- Clear structural reasoning (chain rule, adjoints, VJP),
- Clean and fully vectorised implementations.

Depth of understanding and clarity of reasoning are valued alongside correctness.